

AI Check for Traceability Report

Pipes in the Desert - Team A

Working Code vs Previous Project Documents

AI Tool used: ChatGPT

1. Scope and sources checked

The traceability check was performed between the current working code on the GitHub master branch and the previous project documents. The repository was treated as the source of truth for the working code. The newest uploaded GUI Program document was also included because it is newer and not yet stored in the repository. The GUI Program document confirms the final source files, including FinalGuiMain.java, System.java, Pipe.java, Pump.java, PipeNetwork.java, WaterFlowManager.java, ScoreBoard.java, and the other core model classes. It also defines the Maven compilation and run commands for the final GUI version.

The check covered the following sources:

- working code on GitHub master branch
- GUI Program document
- GUI plans
- prototype program documentation
- detailed prototype plans
- prototype concept document
- skeleton program and skeleton plans
- analysis model version 2
- requirements specification
- problem definition

2. High-level AI findings

Finding 1 - Core game requirements are traceable to final code

The original problem defines a pipe network containing springs, cisterns, pipes, pumps, pump directions, pump breakdowns, leaking or free-ended pipes, plumbers, saboteurs, movement restrictions, and winner determination by water amount.

The final implementation contains the expected model classes:

- Pipe
- Pump
- Spring
- Cistern
- PipeNetwork
- Player
- Plumber
- Saboteur
- ScoreBoard
- GameTimer
- WaterFlowManager
- System
- FinalGuiMain

The final GUI Program document also lists these files as part of the completed implementation.

AI finding: There is strong traceability from the problem definition to the requirements, from the requirements to the object model, and from the object model to the final source code.

Finding 2 - The final GUI is traceable to the GUI specification and final program document

The final GUI Program document confirms that the final graphical version is started from `app.FinalGuiMain`, either through Maven or by running the compiled class directly. It also states that the main menu allows the user to start a new game, read instructions, or exit.

AI finding: The final GUI behavior is traceable to the GUI Program document and the final repository structure.

Finding 3 - Movement rule was corrected compared to earlier feedback

Earlier versions of the project had uncertainty about whether players could move to springs or cisterns. The final behavior is that players move on pipes and pumps only. This matches the project rule that springs and cisterns are network endpoints and interaction targets, but not normal player positions.

AI finding: The earlier movement inconsistency has been resolved in the final implementation.

Finding 4 - Water flow and scoring are implemented, but manual GUI water-flow trigger is a design choice

The requirements say that delivered water and leaked water must be tracked. The final implementation includes WaterFlowManager.java and ScoreBoard.java, which are listed in the final GUI Program document.

The final GUI contains a controlled way to trigger or visualize water flow. This makes the behavior testable and visible.

AI finding: Water-flow scoring is implemented and testable.

Qualification: If the GUI uses a manual Water Flow button, this should be documented as a presentation/testing control rather than as the only possible final-game interpretation.

3. Requirements Traceability Matrix

Requirement / Feature	Source Rationale	Analysis / Plan Link	Final Code Evidence	GUI / Test Evidence	Status
Start game / setup	Requirements define entering names, choosing roles, choosing duration, and starting the game.	Start Game / Supervise Game use case	FinalGuiMain.startGame(), System.startGame(), team setup, PipeNetwork, ScoreBoard, GameTimer	Start Game screen and setup dialog	Traceable
Team size	Original game setup requires teams with	Start / Setup Game	GUI setup validates Plumber and Saboteur team sizes.	Invalid setup is rejected.	Traceable

Requirement / Feature	Source Rationale	Analysis / Plan Link	Final Code Evidence	GUI / Test Evidence	Status
	enough players.				
Initial network	Requirements require a starting network with spring, cistern, pump, and pipes.	Start Game / Initialization	System, PipeNetwork, Spring, Cistern, Pump, Pipe	Initial board contains spring, pump, pipes, and cistern.	Traceable
Move player	Players move on pipes and pumps.	Move Player	Player.moveTo() validates movement target, occupancy, adjacency, and turn state.	GUI Move action selects pipe or pump.	Traceable
Pipe occupancy	Only one player can stand on a pipe; multiple players can stand on a pump.	Move Player / Pipe state	Pipe.occupiedBy, setOccupant(), clearOccupant(), Player.setPosition()	Movement restriction visible in GUI.	Traceable
Pump direction	Pump has one input and one output.	Change Pump Direction	Pump.setDirection(), activeInput, activeOutput	Change Pump Direction GUI action	Traceable
Pump breakdown	Pumps can go out of order.	Randomly Break Pump / Pump state	Pump.breakDown(), Pump.repair()	Break Pump action for demonstration/testing	Partly traceable
Repair pump	Plumbers fix broken pumps.	Repair Pump	Plumber.repairPump(), Pump.repair()	Repair Pump GUI action	Traceable

Requirement / Feature	Source Rationale	Analysis Plan Link	Final Code Evidence	GUI / Test Evidence	Status
Puncture pipe	Saboteurs puncture pipes.	Puncture Pipe	Saboteur.puncturePipe(), Pipe.puncture()	Puncture Pipe GUI action	Traceable
Repair pipe	Plumbers repair leaking pipes.	Repair Pipe	Plumber.repairPipe(), Pipe.repair()	Repair Pipe GUI action	Traceable
Free pipe end disconnect	Pipe ends can be disconnected; free ends leak.	Disconnect Pipe	Pipe.disconnectEnd(), PipeNetwork.disconnectElements()	Disconnect Pipe GUI action	Traceable
Connect pipe	Free pipe ends can reconnect to elements.	Connect Pipe	Pipe.connectFreeEnd(), PipeNetwork.connectElements()	Connect Pipe GUI action	Traceable
New pipes from cistern	Cisterns manufacture or provide pipes.	Generate New Pipe / Place Pipe	Cistern.generatePipe(), Plumber.collectPipeFromCistern(), placeNewPipe()	Add Pipe / material generation GUI behavior	Traceable
New pumps from cistern	Cisterns manufacture or provide pumps.	Generate New Pump / Insert Pump	Cistern.generatePump(), Plumber.collectPumpFromCistern(), PipeNetwork.insertPump()	Add Pump GUI behavior	Traceable
Water delivery leakage scoring	Plumbers maximize delivered water; saboteurs maximize leaked water.	Water Flow / ScoreBoard	WaterFlowManager, ScoreBoard	Water Flow action updates scores.	Traceable

Requirement / Feature	Source Rationale	Analysis Plan Link	Final Code Evidence	GUI / Test Evidence	Status
End game / winner	Team with better water result wins.	End Game / Declare Winner	ScoreBoard.determineWinner(), GameTimer, final screen logic	End Game / final score screen	Traceable
GUI visibility	UI should show network, players, scores, timer, and actions.	GUI plans	FinalGuiMain, panels, HUD, toolbar, map renderer	Final GUI screen	Traceable

4. Bidirectional traceability notes

4.1 Document → Code traceability

Most documented requirements have direct code implementation.

Examples:

- Start/setup game → FinalGuiMain.startGame(), System.startGame()
- Move player → Player.moveTo()
- Pipe occupancy → Pipe.setOccupant(), Player.setPosition()
- Pipe puncture/repair → Pipe.puncture(), Pipe.repair()
- Pump direction/break/repair → Pump methods and GUI toolbar actions
- Network changes → PipeNetwork.connectElements(), disconnectElements(), insertPump()
- Flow and score → WaterFlowManager, ScoreBoard
- GUI → FinalGuiMain, Swing panels, toolbar, HUD, map renderer

This direction is mostly provable.

4.2 Code → Document traceability

Most final code features also trace backward to documents. We have also added a detailed “reverse traceability check” at the end of this document.

Examples:

- Domain classes match the analysis and detailed-plan object catalogue.
- GUI actions match the GUI plan and final README behavior.
- The final program document lists the implemented files and deployment guide.
- The final implementation keeps the same main domain concepts used throughout the semester.

This direction is mostly provable.

5. Cases where bidirectional traceability is not fully provable

5.1 Manual actions that are outside of the scope of problem definition

The original problem does not mention actions such as “Break Pump”, “WaterFlow”, “Remove Pipe”, “Remove Pump”. We have added these as testing activities. They should not be confused as actual gameplay mechanics. AI has found this and says:

Classification: acceptable GUI/testing adaptation, but it should be documented as a controlled demonstration action.

Recommendation: Mark these as optional GUI/testing utilities, not core problem requirements.

Our team agrees with this recommendation. They are indeed not core problem requirements and they can be removed from the program in the future.

5.2 Exact visual layout cannot be fully proven from old documents

The final GUI contains a full Swing interface with menus, game board, player interaction controls, score/timer displays, and status information. The final GUI Program document confirms the presence of FinalGuiMain.java and the final source files, but it does not fully prove every exact visual position, color, or layout decision.

Classification: partially traceable.

Functional GUI traceability is strong, but exact visual traceability depends on whether the GUI plan document contains matching screenshots/mockups.

As a team, we have decided to not use the layout we planned for the game. For that reason, full traceability is not possible.

5.3 Historical prototype tests vs final GUI behavior

Earlier prototype test resources and plans do not automatically prove every final GUI action unless final test protocols explicitly map them to the final GUI.

Classification: not fully bidirectionally provable unless final test records are updated.

6. Inconsistencies / outdated elements

6.1 Old skeleton/prototype documents may still contain skeleton-style prompts

Earlier skeleton/prototype documents used yes/no prompts for random/system events. In the final GUI, some behavior is integrated into GUI controls, timers, or toolbar actions.

Priority: Medium

Clarification: Skeleton prompts were replaced by GUI controls and automatic/timed behavior in the complete program.

6.2 Timer-based ending is a team design choice

The original problem focuses mainly on the team collecting or leaking the most water. Later requirements and the final GUI use a game duration/timer.

Priority: Low

Reason: This was treated as a project customization during development and is visible in later project documents.

6.3 Some final GUI actions are broader than original problem scope

Actions such as Remove Pipe, Remove Pump, or manual Water Flow are useful for presentation/testing, but they are not direct original problem actions.

Priority: Low

Recommendation: Label these as GUI/testing utilities or presentation controls.

Our team agrees with the recommendation.

7. AI findings I would agree / not fully agree with

“Random breakdown is missing” - I would partially disagree

The final program has a controlled Break Pump action and model support for broken pumps. That proves pump breakdown behavior exists. However, if the teacher expects automatic random pump failure, then the trace is only partial.

“The final GUI duplicates model logic” - I would partially agree

FinalGuiMain is large and contains controller behavior, GUI state, and some visual/game coordination. However, the main domain rules still live in model classes such as:

- Player
- Pipe
- PipeNetwork
- Pump
- WaterFlowManager
- ScoreBoard

Therefore, this is an architecture compromise, not a fatal inconsistency.

8. Summary for presentation

The AI traceability check found that the final working code is mostly consistent with the project documents.

The strongest traceability links are:

- Problem definition → requirements → model classes
- Requirements → use cases → final GUI actions
- Analysis object model → Java classes
- Detailed plans → implementation classes
- GUI plan → final FinalGuiMain
- Prototype logic → final GUI behavior

The most important fully traceable features are:

- start game
- player/team setup

- movement on pipes and pumps
- one player per pipe
- multiple players on pumps
- pipe puncture and repair
- pump repair and direction change
- pipe connect/disconnect
- pump insertion
- component pickup/placement
- water delivery/leakage scoring
- winner declaration
- GUI display of map, players, scores, timer, and actions

The main traceability limitations are:

- manual Break Pump is not the same as random pump breakdown
- manual Water Flow button is a controlled simulation step, not necessarily automatic continuous flow
- Remove Pipe / Remove Pump are extra GUI/testing features
- exact visual layout is not fully provable from older documents
- historical prototype tests do not prove every final GUI action unless updated test records exist

Overall, the AI finding is that the project is traceable enough for final submission, with a few acceptable evolution points from skeleton/prototype to complete GUI program.

AI-Based Reverse Traceability Check

Pipes in the Desert - Team A

Final Code → Project Documents

1. Purpose

This traceability check starts from the implemented final program and traces the main classes, GUI actions, and system features backward to the previous project documents.

The goal is to check whether the final implementation is justified by earlier artifacts such as the problem definition, requirements, analysis model, prototype documents, GUI plans, and final GUI program document.

This report checks whether the implemented code/features can be traced backward to:

- problem definition
- requirements documentation
- analysis model
- skeleton/prototype documents
- GUI plans
- final GUI program document
- README / repository behavior description

2. Reverse Traceability Matrix

Implemented code / feature	What exists in final code	Backward trace to documents	Status
FinalGuiMain.java	Final Swing GUI entry point and main graphical application	Traces to GUI Plans document and GUI Program document. The final GUI Program document lists FinalGuiMain.java as the class that starts and manages the final GUI program.	Traceable
Maven project structure	pom.xml, Maven build/run workflow	Traces to final deployment guide. README states final GUI is run with mvn clean compile and mvn exec:java.	Traceable
Java Swing GUI	Full graphical interface instead of console-only interaction	Traces to Task 11 GUI specification and Task 13 Complete Program. README states the game opens in a 1920×1080 Java Swing window.	Traceable

Implemented code / feature	What exists in final code	Backward trace to documents	Status
Main menu	Start Game, Instructions, Exit	Traces to GUI plan and final GUI program run description. The final document states the main menu allows starting a new game, reading instructions, or exiting.	Traceable
Setup screen	Four-player setup, player names, roles, duration	Traces to requirements and prototype setup use cases. README says the user starts a game and configures four players.	Traceable
Team.java	Stores team members	Traces to analysis model and detailed prototype plans where Team groups players into Plumbers and Saboteurs. Final GUI Program lists Team.java as storing team members.	Traceable
Player.java	Base player class, movement, position, turn state	Traces to analysis model and Move Player use case. Final GUI Program lists Player.java as the base class for players.	Traceable
Plumber.java	Plumber-specific actions	Traces to plumber use cases: repair pipe, repair pump, connect/disconnect pipe, add pipe, add pump. Final GUI Program lists Plumber.java as handling plumber actions.	Traceable
Saboteur.java	Saboteur-specific actions	Traces to saboteur use cases: puncture pipe and disrupt flow. Final GUI Program lists Saboteur.java as handling saboteur actions.	Traceable
NetworkElement.java	Base class for physical elements	Traces to class model inheritance where Pipe, Pump, Spring, and Cistern are network elements. Final GUI Program lists it as the base class of pipe-system elements.	Traceable

Implemented code / feature	What exists in final code	Backward trace to documents	Status
Pipe.java	Pipe state, puncture, repair, occupancy, water, free end	Traces to problem definition and pipe-related use cases. README describes pipes as movement targets, connectable/disconnectable elements, and leak sources.	Traceable
Pump.java	Pump state, repair, direction, repair, breakdown	Traces to pump direction, pump repair, pump breakdown requirements. README describes Change Pump Direction, Repair Pump, and Break Pump behavior.	Traceable
Spring.java	Produces water for network	Traces to problem definition: springs are water sources. Final GUI Program lists Spring.java as producing water for the network.	Traceable
Cistern.java	Stores delivered water and materials	Traces to problem definition: cisterns receive water and provide new pipes/pumps. Final GUI Program lists Cistern.java as handling cistern behavior and stored materials.	Traceable
PipeNetwork.java	Stores and updates graph topology	Traces to class model and all network-modification use cases. Final GUI Program lists it as storing and updating the pipe network.	Traceable
WaterFlowManager.java	Calculates water flow and score changes	Traces to water leakage/flow logic use case. Final GUI Program lists it as calculating water flow and score changes.	Traceable
ScoreBoard.java	Stores plumber/saboteur scores	Traces to scoring and end-game use cases. README states that connected/repaired paths score for Plumbers and punctured/free-end paths score for Saboteurs.	Traceable

Implemented code / feature	What exists in final code	Backward trace to documents	Status
GameTimer.java	Countdown timer	Traces to later requirements/prototype decisions about game duration. Final GUI Program lists it as handling the game countdown timer.	Traceable, but design-added
System.java	Game setup, turns, state management	Traces to system-controller responsibilities in analysis and detailed plans. Final GUI Program lists it as controlling setup, turns, and game state.	Traceable
Main.java	Console prototype entry point	Traces to prototype program phase. Final GUI Program still lists it as the console/prototype menu version.	Traceable but legacy
Assets / images / fonts	Pixel-art GUI visuals	Traces to GUI plan and final GUI presentation needs. README lists assets/images and assets/fonts.	Traceable
Scripts	Helper scripts for run/test	Traces to deployment/testing convenience. README lists scripts and PowerShell helpers.	Traceable

3. Features with partial or weaker traceability

Some implemented GUI features are useful, but they are not directly described in the original problem statement.

Feature	Reason for partial traceability	Evaluation
Break Pump button	Original problem describes random/system pump breakdown, not manual GUI action	Acceptable as testing/demo control
Water Flow button	Flow is required, but manual triggering is a GUI design choice	Acceptable as controlled simulation step

Feature	Reason for partial traceability	Evaluation
Remove Pipe / Remove Pump	Original problem focuses on disconnecting/inserting, not removing components	Should be treated as GUI/testing utility
Large FinalGuiMain class	GUI plan expects separation, but final implementation combines GUI/controller logic	Acceptable implementation compromise

4. Reverse traceability verdict

Most final code elements can be traced back to earlier documents. The strongest traceability links are:

- domain classes → analysis model and detailed plans
- GUI actions → use cases and GUI plans
- movement and occupancy → problem definition
- water flow and scoring → requirements and prototype logic
- final deployment → GUI Program document

The main weak points are not missing functionality. They are mostly extra GUI controls added for easier testing, demonstration, or presentation.

These features should be explained as implementation utilities rather than core game rules.

5. Final conclusion

The final implementation is mostly traceable to the previous project documents.

The code supports the main game model, player actions, pipe-network behavior, scoring logic, and GUI interaction planned throughout the semester. Some final GUI actions go beyond the original requirements, but they are acceptable if documented as testing or demonstration utilities.